ECE449 Computer architecture Project explain 1

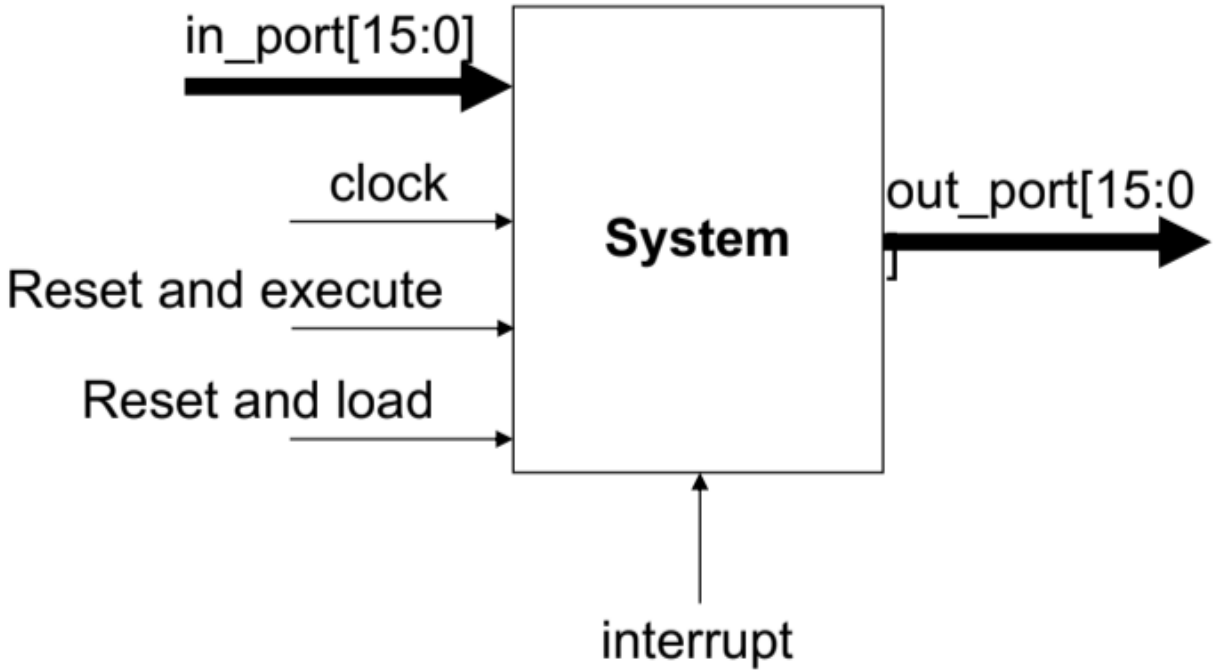
Lehung Nguyen

Reference

- ECE499_Project.pdf
- ProjectPresentation.pdf
- Basys 3 reference manual:
  - https://digilent.com/reference/programmable-logic/basys-3/reference-manual?srsltid=AfmBOoryXWaraevJBYB0y59GFjss0ZI9fc3AvaOE-CWx_SDR_5BmwsUy

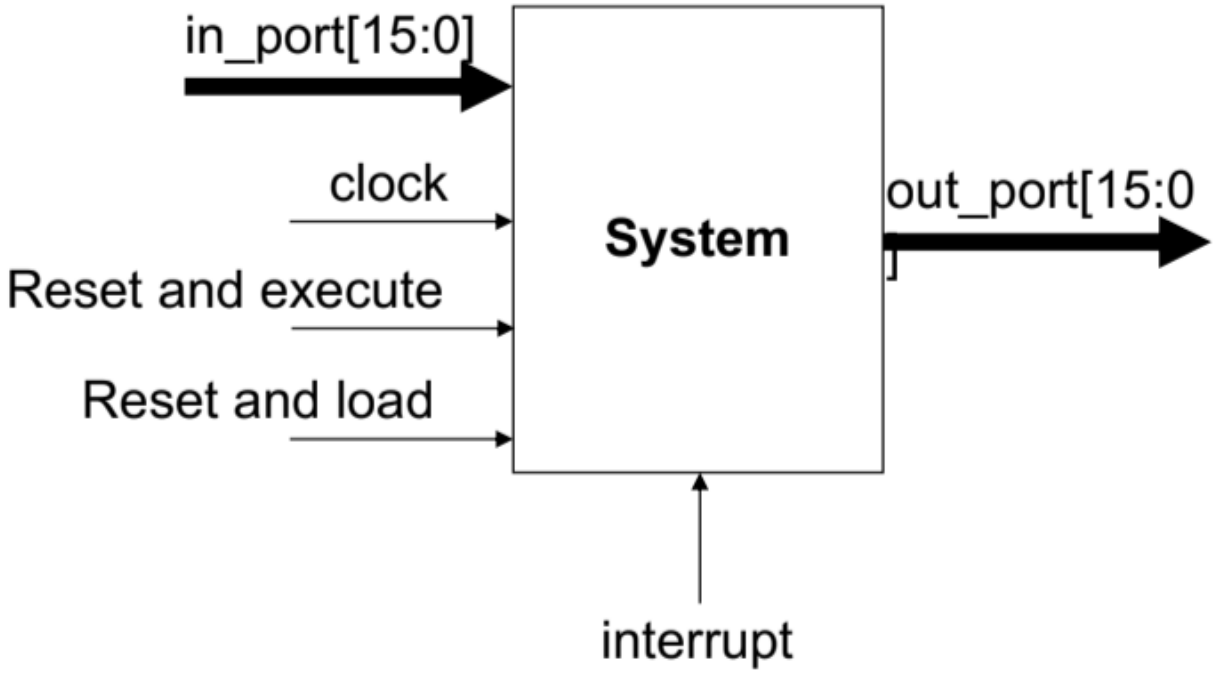
Pinout of Processor

- *In_port* and *out_port* are critical because they are the *only interface* your CPU has with the outside world
  - Without them, CPU could calculate values internally, but you would never see the results or be able to provide input
- *Physical Connection*: These ports are 16-bit wide buses ([15:0])
  - *in_port*: Connects to the 16 switches on the board, allowing you to manually input binary data
  - *out_port:* Connects to the 16 LEDs, allowing you to visualize the result of your calculations.



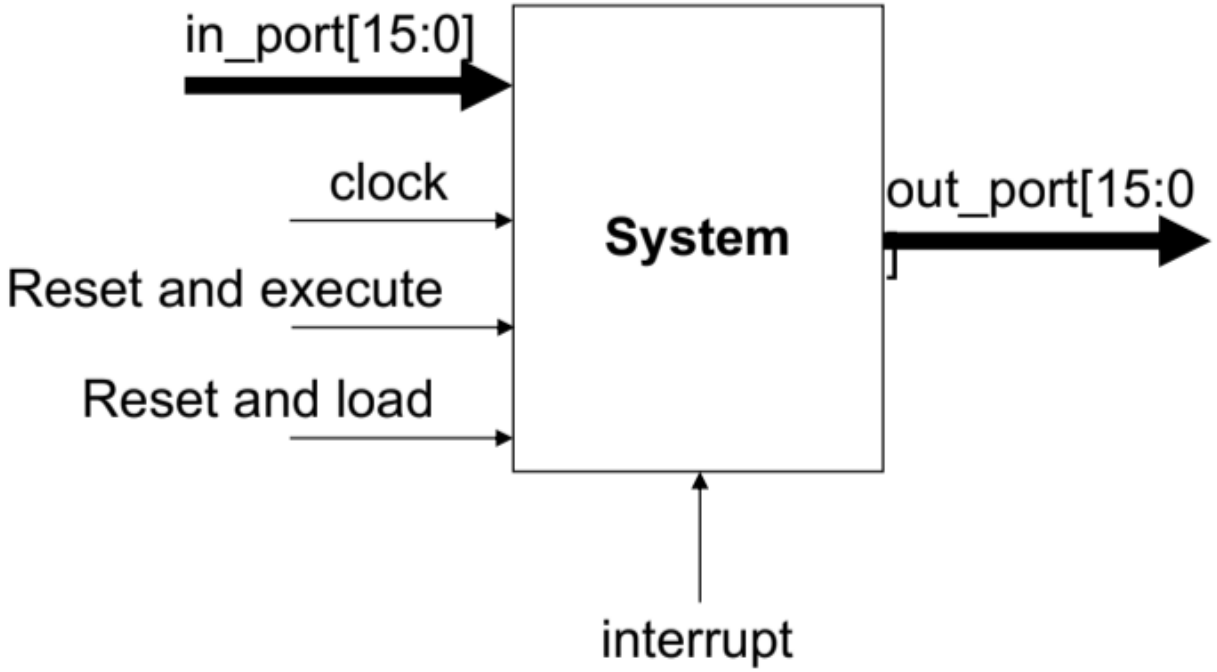
Memory mapping

- *Memory Mapping*: these ports are "Memory Mapped," meaning the CPU treats them like specific memory addresses
  - *input port* is mapped to address *0xFFF0*
  - *output port* is mapped to address *0xFFF2*
- E.g.: To read a switch, your code simply loads from address *0xFFF0*. To light up an LED, your code "stores" a value to *0xFFF2*



Two reset modes

- Two distinct reset buttons: *"Reset and execute"* and *"Reset and load"*
- The difference lies in *where* the CPU starts running code (the reset vector) and *what* that code does



	Reset & Execute	Reset & Load
Purpose	Runs the user program currently stored in RAM.	Enters a mode to accept *new* code and write it into RAM.
Vector Address	Jumps to 0x0000.	Jumps to 0x0002.
Behavior	The BIOS at 0x0000 immediately branches to the start of your application.	The BIOS at 0x0002 runs a "loader" routine that listens for input (likely via in_port) and writes it to memory

Why reset modes matter

- These modes are essential because they implement a rudimentary *BIOS (Basic Input/Output System)*.
  - *"Empty Brain" Problem:* When you power on the FPGA, the RAM is volatile (empty or random, requires constant power to retain data). The CPU needs a way to get a program *into* that RAM.
  - *Avoids Hardware Re-synthesis:* Without the "Reset and Load" feature, you would have to "hardcode" your test program into the hardware design and re-synthesize the entire FPGA (which takes 10+ minutes) every time you changed a single line of assembly code.
- Workflow:
  - Press *Reset and Load:* The CPU waits. You feed it your machine code (via switches or external interface). The BIOS writes this code into RAM.
  - Press *Reset and Execute:* The CPU jumps to the code you just loaded and runs it.

Interrupt is Optional

- "Interrupt is optional" significantly defines the scope of your project. It is important for two main reasons
  - *Complexity Reduction:* Implementing interrupts *adds substantial complexity to both the hardware and the controller. You must handle asynchronous events (events that happen at random times), save the current state of the processor (Context Switching), and restore it perfectly later.* By making it optional, the project allows you to build a functioning "polling-based" CPU first without worrying about these difficult timing and storage issues.
  - *Pin Management:* The pinout on Slide 2 shows an *interrupt* pin. If you choose not to implement this feature, you can simply tie this input to *0 (ground)* in your top-level module and ignore it in your control logic.

Optional: How to Change Code and Design to Implement *Interrupts*

- If you choose to implement the optional interrupt, you must modify your design to support *Stack Operations* and *Context Switching*. Based on the "Optional" instruction list in ***ProjectPresentation24.pdf*** **(Slide 13)**

16-bit (Optional)

Mne-monic	Op-code	Function		
PUSH	96	M[SP--] ← R[ra];	A3	*PUSH ra*
POP	97	R[rb] ← M[++SP];	A3	*POP ra*
LOAD.SP	98	SP←R[ra];	A3	*LOAD.SP ra*
RTI	99	PC ← M[++SP]: {Z,N} restored	A0	*RTI*

Optional: How to Change Code and Design to Implement Interrupts

- 1. Hardware Changes (Datapath)
  - *Add a Stack Pointer (SP):* You need a dedicated register (often distinct from r0-r7) to track the top of the stack in memory
  - *Modify Datapath for Stack Access:* The ALU and Memory stages must be able to use the SP as an address source, not just *ra, rb, or rc*
- 2. Controller & FSM Changes
  - Interrupt Detection*:* The Controller's Finite State Machine (FSM) must check the *interrupt* input signal at the end of every instruction cycle (usually before the *Fetch* stage).

Optional: How to Change Code and Design to Implement Interrupts

- 2. Controller & FSM Changes
  - Interrupt Detection*:* The Controller's Finite State Machine (FSM) must check the *interrupt* input signal at the end of every instruction cycle (usually before the *Fetch* stage).
  - *Hardware Vectoring:* If the interrupt signal is high:
    - *Stall* the fetching of the next user instruction.
    - *Auto-Push State*: The controller must sequence a hardware operation to write the current *Program Counter (PC)* and *Flags (Z, N)* to memory at the location pointed to by SP.
    - *Vector Update:* Force the PC to a specific "Interrupt Vector Address" (e.g., a specific line in the BIOS/ROM) to start the Interrupt Service Routine (ISR).

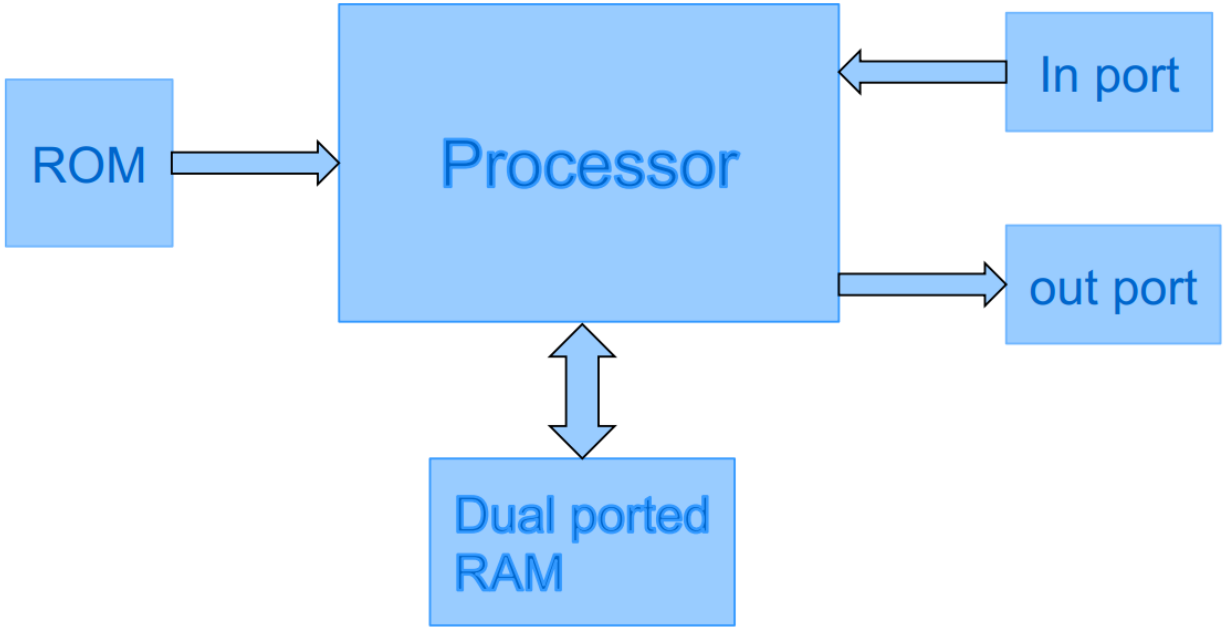
Optional: How to Change Code and Design to Implement Interrupts

3. New Instructions Required

- You must implement the optional instructions listed in ***ProjectPresentation24.pdf*** **Slide 13** to manage the saved state:
  - PUSH (Opcode 96): To manually save registers (r0-r7) onto the stack so the ISR can use them without deleting user data.
  - POP (Opcode 97): To restore those register values before returning.
  - RTI (Return from Interrupt, Opcode 99): A specific instruction that pops the saved *PC* and *Flags* from the stack back into the processor registers, resuming the original program exactly where it left off.
- Summary of Workflow Change: your CPU strictly follows
  - Without interrupts: Fetch -> Decode -> Execute
  - With interrupts: Fetch -> Decode -> Execute -> [Check interrupt] -> (If True: Save PC, Jump to ISR)

System Description

- *Processor:* CPU controls the data flow and executes instructions.
- *ROM (Read-Only Memory):* a small, permanent memory block containing the "BIOS" (Basic Input/Output System). Processor can only *read* from it, not write to it (One-way arrow)
  - BIOS code is provided
- *Dual Ported RAM*: main system memory to store user programs and data. processor can both *read* and *write* data here (two-way arrow).
- *I/O Ports: In port* and *out port* connect to the outside world (switches and LEDs). The processor reads from the input and writes to the output.



RAM & Harvard architecture

- Memory design dictates how your pipeline works

1. Dual Ported RAM & Harvard Architecture
   - *Von Neumann Bottleneck*: In a standard simple computer (Von Neumann), there is only one door to memory. The CPU cannot fetch an *instruction* (to know what to do) and load *data* (to actually do it) at the exact same time. It has to wait.
   - *The Solution (Harvard Architecture):* True Harvard architecture uses two physically separate memory chips: one for Instructions and one for Data.
   - *Your Implementation:* You are simulating a Harvard Architecture using a single memory block with two "doors" (ports).
     - *Port A (Data):* Used by the Memory Stage of your pipeline to LOAD or STORE data. It supports both reading and writing.
     - *Port B (Instructions):* Used by the Fetch Stage of your pipeline to grab the next instruction. It is typically Read-Only.
     - This allows your pipeline to *fetch instruction N+2 (via Port B)* at the exact same moment it is *saving the result of instruction N (via Port A)*.

RAM Serializes access requests

- The RAM serializes the access requests arbitrarily
- While the ports *look* independent to you, inside the physical FPGA memory block, there is logic handling the requests. If you try to *write to Address X (via Port A)* and *read from Address X (via Port B)* on the exact same clock cycle, the hardware has to decide which happens first.
- The "arbitrary" serialization means you cannot guarantee which happens first in a collision. However, in a standard pipeline, you *rarely write to the exact address you are currently executing* (unless implementing self-modifying code), so this is rarely a fatal issue.

RAM Implementation (Synchronous)

- Use the dual port distributed *RAM macro XPM_MEMORY_DPDISTRAM* from Xilinx XPM macro group
- *Synchronous Behavior:* This RAM is *synchronous*. This means it requires a clock signal to work.
  - *Asynchronous (Bad for FPGAs):* You send an address, and data appears instantly (like a combinational circuit).
  - *Synchronous (Required): You* send an address on the rising edge of the clock, and the data appears on the *next* clock cycle (or after a specified delay).
- You must account for this delay in your pipeline. E.g. If you ask for an instruction in Cycle 1, *it will not be available for the Decode stage until Cycle 2*. The *XPM macro* allows you to specify this latency (delay) explicitly.

Read-Only Memory (ROM) and the Basic Input/Output System (BIOS)

- CPU faces a fundamental problem when it is first powered on:
  - its main memory (RAM) is empty (or contains random garbage data).
  - If the CPU tried to fetch instructions from RAM immediately, it would crash.
  - *Solution:* The ROM provides a small, permanent storage space for startup code
  - *Non-Volatile:* Unlike RAM, *ROM keeps its data even when power is cut*. This ensures that every time you turn on the board, *there is valid code at address 0x0000 for the CPU to start executing immediately*.
- Can we modify the BIOS?
  - ROM is *read-only* to the processor. CPU can fetch instructions from it, but it cannot write new data into it.
  - You cannot update the BIOS while the CPU is running. To "change" the BIOS, you would have to modify the initialization file (usually a .coe or .mem file) in your Vivado project and *re-synthesize* the entire hardware design.
  - In this project, *ROM and its BIOS will be provided to you*. Do not need to modify it; just need to *ensure CPU correctly fetches instructions from it*

Two main BIOS functions

BIOS solves the "empty RAM" problem by acting as a bridge

- Function 1: Load User Code
  - Your specific program ("User Code") isn't on the chip yet.
  - press *Reset and Load* (which vectors to 0x0002), the CPU executes BIOS instructions that:
    - Read data from the *Input Port* (connected to switches or a keypad)
    - Write that data into *RAM* starting at address 0x0400
  - Result**:** This process manually copies your program into the computer’s memory.
- Function 2: Execute User Code
  - Problem: The CPU is currently running BIOS code in ROM (0x0000 range), but your program is in RAM (0x0400 range).
  - BIOS Action: When you press Reset and Execute, the BIOS performs a "Branch" (Jump) instruction to address 0x0400.
  - Result: The CPU stops fetching from ROM and starts fetching your instructions from RAM.

User’s code & Instruction

- User's Code: refers to the machine language program (1s and 0s) that *you* want your custom CPU to run. E.g, if you want your **CPU to calculate *5 + 3***, the "User Code" is the binary sequence for *LOAD r1, #5; LOAD r2, #3; ADD r3, r1, r2*.
- First, the CPU executes the *BIOS* (instructions stored in ROM) to set things up. Then, the CPU executes *User Code* (instructions stored in RAM) to do your actual work.
- User Code (or a "program") is simply the *sequence* of these instructions put together to solve a specific problem. If an "Instruction" is a word, then "User Code" is a sentence.
- The BIOS doesn't care what the sentence says; its only job is to copy the whole paragraph (the User Code) from the outside world into the RAM so the CPU can read it later.

User's code and instruction

- Instruction: a single, atomic command that the CPU understands. It is the smallest building block of a program.
  - In this project, use set of commands defined in **ECE449_project.pdf Slide 10** (A-Format, B-Format, L-Format).
- For the "5 + 3" example, you cannot just tell the CPU "calculate 5+3". You must break it down into specific *instructions* that match your CPU's hardware design:
  - *Instruction 1:* Move the number 5 into Register 1.
  - *Instruction 2:* Move the number 3 into Register 2.
  - *Instruction 3:* Add Register 1 and Register 2, putting the result in Register 3.
- Each of these lines is one *instruction*.

Instruction Format

- Three types of instructions
  - A-Format
    - e.g. arithmetic instructions
  - B-Format
    - e.g. branch instructions
  - L-Format
    - e.g. load and store instructions

Example

- Human thoughts: Calculate 5+3
- list of instructions using the mnemonics from **ECE499_Project.pdf Slide 17** (L-Format) and **Slide 11** (A-Format):
  - LOADIMM r1, #5 (Load Immediate value 5 into register 1)
  - LOADIMM r2, #3 (Load Immediate value 3 into register 2)
  - ADD r3, r2, r1 (Add r2 and r1, store in r3)
- Machine Code (The Actual "User Code"): The CPU doesn't read text; it reads binary. An assembler must convert the instructions above into 16-bit binary numbers

Instruction	Format	Binary Code (Hypothetical based on Slide 11/17)	What is it?
LOADIMM r1, #5	L-Format	10010 001 00000101	**Instruction 1**
LOADIMM r2, #3	L-Format	10010 010 00000011	**Instruction 2**
ADD r3, r2, r1	A-Format	0000001 011 010 001	**Instruction 3**

Example

- "User Code" is the entire file containing these three binary lines
  - 1001000100000101
  - 1001001000000011
  - 0000001011010001
- *Reset and Load:* You feed that block of three binary lines (the User Code) into the CPU through the Input Port. The BIOS saves it into RAM at address 0x0400.
- *Reset and Execute:* The CPU looks at address 0x0400, grabs the first sequence of bits (1001...), decodes it as "Load 5", and executes that *instruction*. Then it moves to the next one.

Resets

- Vectors to address = refers to the hardware automatically forcing the *Program Counter (PC)* to point to a specific memory address.
  - Program Counter (PC) is a finger pointing to the line of code the CPU is currently reading.
  - When you press a Reset button, the hardware physically grabs that finger and forces it to point to a specific line number (address) at the very beginning of the memory map.
  - press *Reset/Execute*, the hardware forces the PC to point to *0x0000*.
  - press *Reset/Load,* the hardware forces the PC to point to *0x0002*.

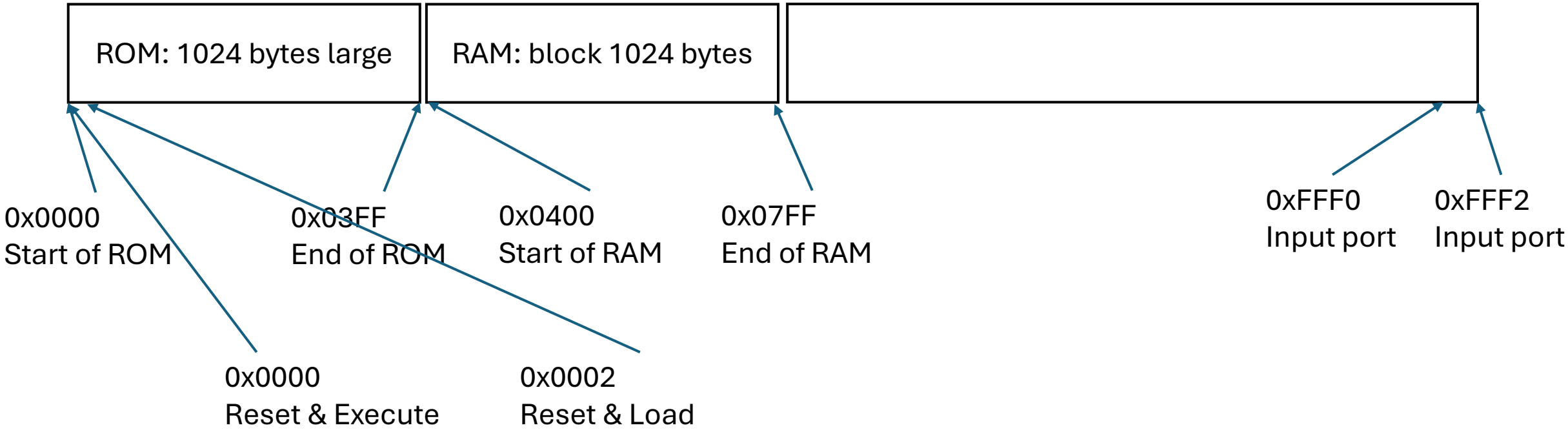
Resets

- At each address, the developer has introduced the appropriate branch (BRR) instruction that vectors to the reset-handling routine (this is part of the BIOS)

Address	Instruction Stored There	What it does
0x0000	BRR +0x100 (Example)	**"Reset/Execute" lands here.** This instruction immediately says: " *Jump way down to address 0x0100 where the execution code lives."*
0x0002	BRR +0x200 (Example)	**"Reset/Load" lands here.** This instruction immediately says: *"way down to address 0x0200 where the loader code lives."*
...	...	(Empty space or other data)
0x0100	MOV...	**ASSUME**: The actual code for running the user program starts here (safe from collisions).
0x0200	IN...	**ASSUME**: The actual code for loading new data starts here (safe from collisions).

ROM, RAM, and Ports

- Memory map: tells CPU which physical component lives at which address



Instruction Format

- Three types of instructions
  - A-Format
    - e.g. arithmetic instructions
  - B-Format
    - e.g. branch instructions
  - L-Format
    - e.g. load and store instructions

- A-Format: Arithmetic Instructions
  - focus on *Registers*. They typically take two source registers, perform math, and store the result in a destination register
- B-Format (Branch Instructions)
  - focus on the *Program Counter (PC)*. Instead of manipulating data, they manipulate *where* the CPU looks for the next instruction. They use a "Displacement" (offset) instead of just registers.
- L-Format (Load/Store Instructions)
  - bridge the gap between the CPU and *Memory (RAM)*. They often contain an "Immediate" value (a raw number) or a memory address.

Feature	**A-Format**	**B-Format**	**L-Format**
Primary Job	**Calculation** (Math/Logic)	**Control Flow** (Jumping)	**Data Movement** (Memory Access)
Main Target	ALU (Arithmetic Logic Unit)	Program Counter (PC)	Data Memory (RAM) or Register File
Operands	Registers (r0-r7)	Displacement (Offets)	Addresses & Immediate Values
Example	ADD, SUB, NAND	BR (Branch), BR.SUB	LOAD, STORE, LOADIMM

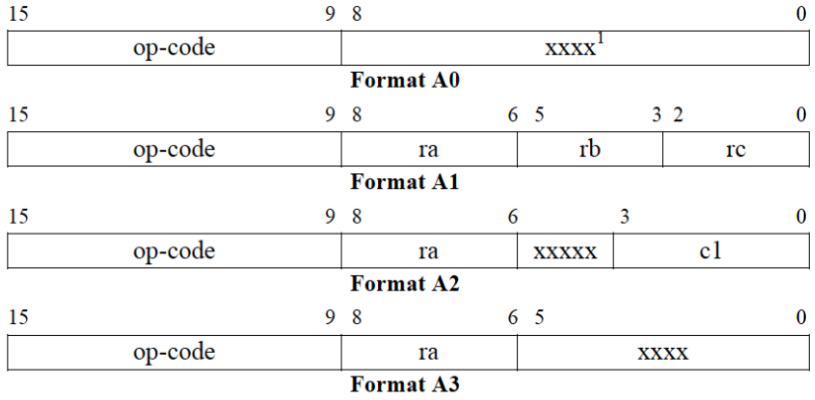
A format

- Structure: These instructions focus on Registers. They typically take two source registers, perform math, and store the result in a destination register.
- Key Fields
  - Op-code (7 bits): Tells the CPU what math to do (e.g., ADD, NAND).
  - ra, rb, rc (3 bits each): The addresses of the registers to use.
  - When to use: Use this whenever you need to modify data that is already inside the CPU.
  - Example: ADD r3, r2, r1Logic: Take value in r1, add it to r2, put result in r3.

Binary string: **0000001 011 010 001**

A-Format Instructions

Arithmetic Instructions:



e.g.:

ADD r3,r2,r1

15 9	8 6	5 3	2 0
0000001	**011**	**010**	**001**

Field Name	Bit Range	Binary Value	Decimal Value	Meaning
Op-code	Bits 15-9	0000001	**1**	Tells the CPU to perform an **ADD** operation.
ra	Bits 8-6	011	**3**	Destination Register: **r3**
rb	Bits 5-3	010	**2**	First Source Register: **r2**
rc	Bits 2-0	001	**1**	Second Source Register: **r1**

- Op-code" (Operation Code) is a unique ID number for every command the CPU knows
- Values for Op-code in **ProjectPresentation24.pdf slide 8**

16-bit (A Format)

Mne-monic	Op-code	Function	Type	Syntax
NOP	0	Nothing	A0	*NOP*
ADD	1	R[ra] ← R[rb] + R[rc];	A1	*ADD ra,rb,rc*
SUB	2	R[ra] ← R[rb] – R[rc];	A1	*SUB ra,rb,rc*
MUL	3	R[ra] ← R[rb] × R[rc];	A1	*MUL ra,rb,rc*
NAND	4	R[ra] ← R[ra] NAND R[rb]:	A1	*NAND ra,rb,rc*
SHL	5	n:=c1<3…0>): (n|0) → (R[ra]<15…0>←R[ra]<15-n…0>#(n@0)): (n=0) → () ;	A2	*SHL ra#n*
SHR	6	n:=c1<3…0>): (n|0) → (R[ra]<15…0>←(n@0)#R[ra]<15…n>): (n=0) → () ;	A2	*SHR ra#n*
TEST	7	(R[ra] = 0) →Z ← 1; else →Z ← 0: (R[ra] < 0) →N ← 1; else →N ← 0;	A3	*TEST ra*
OUT	32	OUT.PORT ← R[ra];	A3	*OUT ra*
IN	33	R[ra] ← IN.PORT;	A3	*IN ra*

Multiplication needs to be refined. This will affect the TEST and the flags

Op-code

Operation	Op-Code (Decimal)	Op-Code (Binary, 7-bit)	Function
NOP	0	0000000	Do nothing.
ADD	1	0000001	Addition.
SUB	2	0000010	Subtraction.
MUL	3	0000011	Multiplication.
NAND	4	0000100	Bitwise NAND.
SHL	5	0000101	Shift Left.
SHR	6	0000110	Shift Right.
TEST	7	0000111	Check for Zero/Negative flags.
OUT	32	0100000	Write to Output Port.
IN	33	0100001	Read from Input Port.

Register address & Memory address

- *ra, rb, and rc* are *3-bit indices* that point to one of the *8* available registers inside the *CPU's Register File*.
  - Since there are only 8 registers (r0 through r7), you only need 3 bits to identify them ($2^3 = 8$).
  - 000 = Register 0
  - 011 = Register 3
  - 111 = Register 7

Feature	Register Address (ra, rb, rc)	Memory Address (RAM/ROM)
Size	**3 bits**	**16 bits**
Location	Inside the CPU (The "Workbench").	Outside the Datapath (The "Warehouse").
Speed	Instant access (Combinational logic).	Slow access (Requires clock cycles/Pipeline stages).
Capacity	Only 8 locations (r0-r7).	65,536 locations (0x0000-0xFFFF).

Register address & Memory address

- *Memory Address:* This is like a *House Address* (e.g., 1234 Maple St). There are thousands of them, and it takes time to drive there to pick something up.
- *Register Address:* This is like which *Pocket* you are using (Left pocket vs. Right pocket). You only have a few pockets, and you can reach them instantly.
- *How they interact*: You often use a *Register* to hold a *Memory Address*.
  - *Example:* LOAD r1, @r2
  - Here, r2 (Register Address 010) holds the value 0x0400.
  - The CPU looks in "Pocket #2" (r2), finds the number 0x0400, and then sends 0x0400 to the memory controller to fetch data from "House #0x0400".

B-Format Instructions

Branch Instructions:

15 – 9	8 – 0
op-code (BRR)	disp.l

Format B1

15 – 9	8 – 6	5 – 3	2 – 0
op-code (BR)	ra	disp.s	

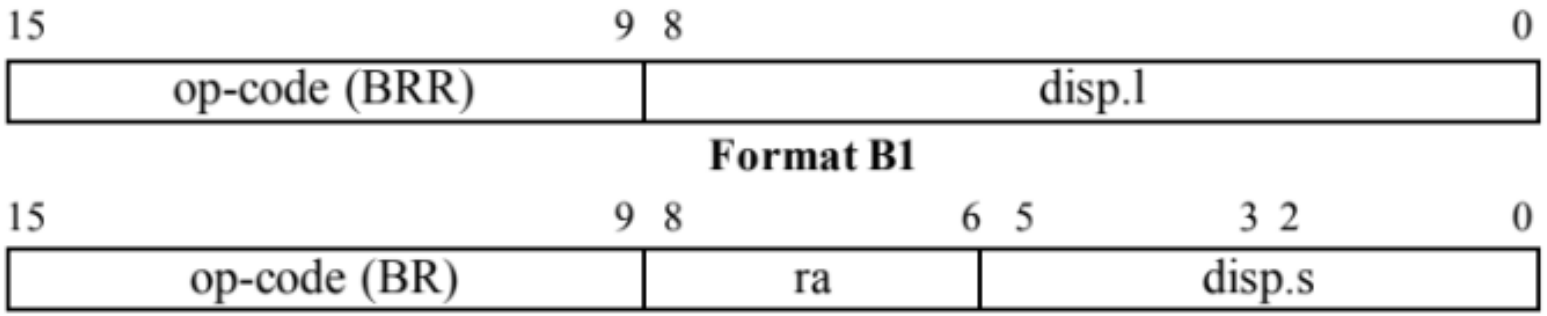
Format B2

e.g.:

BR r3+0x8

15 – 9	8 – 6	5 – 0
1000011	011	001000

Format B (Branch)



16-bit (B Format)

Mne-monic	Op-code	Function	Type	Syntax
BRR	64	PC ← PC+2*disp.l {sign extended 2's complement}	B1	*BRR +disp.l*
BRR.N	65	(N=1) → PC ← PC+2*disp.l {sign extended 2's complement}; (N=0) → PC ← PC+2 { 2's complement};	B1	*BRR.N +disp.l*
BRR.Z	66	(Z=1) → PC ← PC+2*disp.l {sign extended 2's complement}; (Z=0) → PC ← PC+2 { 2's complement};	B1	*BRR.Z +disp.l*
BR	67	PC ← R[ra]+2*disp.s {sign extended 2's complement}	B2	*BR ra+disp.s*
BR.N	68	(N=1) → PC ← R[ra] {word aligned}+2*disp.s {sign extended 2's complement}; (N=0) → PC ← PC+2 { 2's complement};	B2	*BR.N ra+disp.s*
BR.Z	69	(Z=1) → PC ← R[ra] {word aligned}+2*disp.s {sign extended 2's complement}; (Z=0) → PC ← PC+2 { 2's complement};	B2	*BR.Z ra+disp.s*
BR.SUB	70	r7 ← PC + 2; PC ← R[ra] {word aligned}+2*disp.s {sign extended 2's complement};	B2	*BR.SUB ra+disp.s*
RETURN	71	PC ← r7;	A0	*RETURN*

For the PC operations, the argument is the value of the PC just before the instruction is fetched, while the result is the value of the PC at the conclusion of the execution of the instruction.

B-format

- BRR (Branch Relative) - Format B1: CPU takes the current Program Counter (PC) value and adds a "displacement" (offset) to it.
  - Formula: $PC \leftarrow PC \ + \ (2 \times disp.l)$
  - Use this for standard code navigation, like *if* statements or *while* loops (e.g., "Skip the next 5 lines" or "Go back 3 lines").
- BR (Branch via Register) - Format B2: Jump to the address stored inside a specific register. *How it works:* The CPU reads a memory address from a register (ra), adds a small offset (disp.s), and moves the PC there.
  - Formula: $PC \leftarrow R[ra] \ + \ (2 \times disp.s)$
  - Use this for dynamic jumps, such as returning from a subroutine (RETURN is a special type of BR using $r7$) or jump tables.

disp.l and disp.s

- "Disp" stands for *Displacement*. It is a signed integer (positive or negative) that tells the CPU how far to jump.

Feature	disp.l (Displacement Long)	disp.s (Displacement Short)
Format	Used in **Format B1** (BRR).	Used in **Format B2** (BR).
Bit Width	9 bits (Bits 8–0).	6 bits (Bits 5–0).
Range	Can jump farther (approx. ±256 instructions).	Can only jump a short distance (approx. ±32 instructions).
Why the difference?	In Format B1, we don't need to specify a register, so we have more "spare room" in the instruction to fit a larger number.	In Format B2, we need 3 bits to specify the register (ra), so we have less room left for the displacement.

The "Times 2" Rule

- in the formulas that the displacement is always multiplied by 2 ($2 \times disp$).
  - Why: Your memory is byte-addressed, but your instructions are *16 bits (2 bytes) wide*
  - Instructions must always start at an even address (0x0000, 0x0002, 0x0004). You can never jump to address 0x0003
  - By implicitly multiplying by 2, a displacement of "1" tells the CPU to jump "1 whole instruction forward" (2 bytes), rather than "half an instruction forward" (1 byte).
- Example Trace (BRR):
  - *Current PC: 0x0100*
  - *Instruction: BRR +3 (Binary: Opcode + 000000011)*
  - *Calculation:*
    - *Read disp.l = 3.*
    - *Multiply by 2 = 6.*
    - *Add to current PC: 0x0100 + 6 = 0x0106.*
  - *Result: The CPU jumps over the next 3 instructions and lands on the 4th one.*

Subroutine

- *Function Calls* (*Subroutines*) in your CPU. It illustrates how the processor can temporarily leave the main program to perform a specific task and then return exactly to where it left off.
- Instruction *BR.SUB (Branch to Subroutine)*. This is a *B2-Format* instruction.
  - BR.SUB ra + disp (e.g., BR.SUB r2+072).
  - Jump to the memory address calculated by $(Register[r2] + 2 \times 72)$, but remember where we came from.

Subroutine

```
                ...
                ...
                BR.SUB r2+072
                ...

subroutine:     ...
                return
```

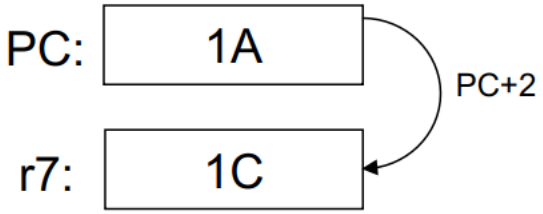
Subroutine

- Register r7 plays the additional role of a Link Register used in subroutine call&return
  - **BR.SUB rx+disp**

The current program counter is placed in r7 while the PC is loaded with the target address (i.e. (rx)+2*disp)

```
        ...
        ...
0X1A:   BR.SUB
0X1C:   ...

subroutine:
        ...
        return
```



For the PC operations, the argument is the value of the PC just before the instruction is fetched, while the result is the value of the PC at the conclusion of the execution of the instruction.

Link register

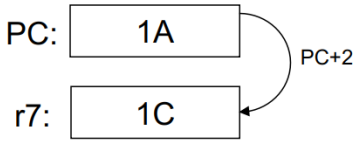
- critical special role of *Register 7 (r7)*.
  - In many simple CPUs, you just jump. But for a subroutine, you need a return ticket.
  - When BR.SUB executes, the CPU automatically saves the address of the *next* instruction (Current PC + 2) into *Register r7*
  - r7 is called the *Link Register (LR)* because it "links" the subroutine back to the main code.
- *Example: Refer to* ***ECE449_Project.pdf slide 13-16***
- *Side Effect*: The CPU looks at the Program Counter (say, 0x1A). It calculates the next address (0x1C) and forces r7 = 0x1C.
- The CPU updates the PC to the target address (the start of the subroutine) and begins executing there
- The CPU runs the code inside the subroutine: block.
- At the end of the subroutine, the code executes the RETURN instruction.
- RETURN is actually just a special nickname for BR r7 (Branch to value in r7).
- The CPU copies the value inside r7 (0x1C) back into the Program Counter.
- Execution resumes immediately after the original BR.SUB call.

- Register r7 plays the additional role of a Link Register used in subroutine call&return
  - **BR.SUB rx+disp**

The current program counter is placed in r7 while the PC is loaded with the target address (i.e. (rx)+2*disp)

```
        ...
        ...
0X1A:   BR.SUB
0X1C:   ...

subroutine:
        ...
        return
```

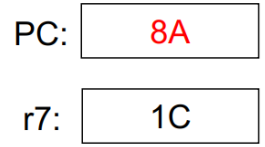


For the PC operations, the argument is the value of the PC just before the instruction is fetched, while the result is the value of the PC at the conclusion of the execution of the instruction.

- LR: a dedicated register for br.sub instructions
  - br.sub: PC+2 is loaded into r7
  - return: PC is loaded with r7

```
        ...
        ...
0X1A:   BR.SUB
0X1C:   ...

subroutine:
        ...
0X88:   return
```

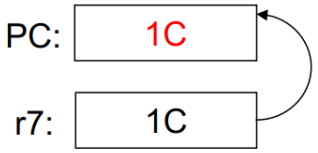


The contents of PC shown are those immediately after return was fetched

- LR: a dedicated register for br.sub instructions
  - br.sub: PC+2 is loaded into r7
  - return: PC is loaded with r7

```
        ...
        ...
0X1A:   BR.SUB
0X1C:   ...

subroutine:
        ...
0X88:   return
```



The contents of PC shown are those immediately after return completed execution

Call a subroutine inside another subroutine

- Because r7 is a single physical register, *you cannot easily call a subroutine inside another subroutine*.
- If Subroutine A calls Subroutine B, the BR.SUB for B will *overwrite* the return address for A stored in r7.
- *Solution*: If you need nested subroutines, you must manually save r7 to memory (using STORE or PUSH) before making the second call.

L-Format

- **Load/Store Instructions**

15 … 9	8	7 … 0
op-code	m.l	imm

Format L.1

15 … 9	8 … 6	5 … 3	2 … 0
op-code	r.dest	r.src	

Format L.2

e.g.: LOAD r1,@r2

or LOAD r.dest,r.src

LOAD r1,r2

15 … 9	8 … 6	5 … 3	2 … 0
0010000	001	010	xxx

L format

- *L-Format* handles moving data between the CPU's registers and the external memory (RAM) or loading constant numbers. Without these instructions, the CPU could calculate things but could never save the results or read new data to work on.
- Two Sub-Formats (L.1 and L.2) two specific structures depending on whether you are using *raw numbers or registers*.
- Format L.1 (Immediate Mode): *Op-code + m.l (1 bit) + imm (8 bits)*.
  - Use: This is primarily used for the LOADIMM instruction.
  - Key Feature: It devotes 8 bits to the imm field, allowing you to load a specific number (like "5" or "100") directly into the upper or lower half of a register.
- Format L.2 (Register Mode): Op-code + r.dest (3 bits) + r.src (3 bits).
  - Use: Used for LOAD, STORE, and MOV instructions.
  - Key Feature: It uses two register indices. One acts as the source of data/address, and the other acts as the destination.

r.dest/r.src and r1/r2

- *r.dest and r.src* (variables)
  - These are *labels* for the specific bit-fields in the binary instruction.
  - *r.dest (Destination):* Bits 8-6. This field tells the CPU: *"Which register should I write the result into?“*
  - *r.src (Source):* Bits 5-3. This field tells the CPU: "Which register holds the data or memory address I need to read?".
- *r1 and r2* (values)
  - These are specific register names used in the example code on the slide (LOAD r1, @r2).
  - In this specific example:
  - r1 is chosen to be the Destination (r.dest).
  - r2 is chosen to be the Source (r.src).
  - Visualizing the Mapping: If you write the code LOAD r1, @r2 ("Load data into Register 1 from the address in Register 2"):

L-format

- two syntax styles for LOAD:
  - *LOAD r1, @r2:* The @ symbol is shorthand for "Address of". This clearly indicates you are using r2 as a pointer to memory.
  - *LOAD r.dest, r.src:* This is the formal definition showing that the first argument is always the destination and the second is the source.
- *STORE* instructions (**ProjectPresentation24.pdf Slide 12**
  - STORE @r.dest, r.src
  - Here, you are taking data from r.src and writing it to memory at the address in r.dest.
  - Note that even though r.dest is holding the address (a source of information), it is still placed in bits 8-6 (the ra position). Be very careful in your Decoder logic to route these wires correctly for STORE

16-bit (L Format)

Mnemonic	Op-code	Function	Type	Syntax
LOAD	16	R[r.dest] ← M[R[r.src]];	L2	*LOAD r.dest, r.src* *LOAD r.drst, @r.src*
STORE	17	M[R[r.dest]] ← R[r.src];	L2	*STORE r.dest, r.src* *STORE @r.dest, r.src*
LOADIMM	18	(m.l=1) →R7<15…8>← imm: (m.l=0) →R7<7…0>← imm;	L1	*LOADIMM.upper #n* *LOADIMM.lower #n*
MOV	19	R[r.dest] ←R[r.src];	L2	*MOV dest,src*